

HỆ THỐNG SELF-HEALING TEST

Tự Phục Hồi Locator Selenium

Tài liệu hướng dẫn sử dụng toàn diện

1. Giới Thiệu Hệ Thống

1.1 Vấn Đề Cần Giải Quyết

Trong quá trình kiểm thử tự động với Selenium, một trong những vấn đề phổ biến nhất là **locator bị hỏng** — xảy ra khi developer thay đổi UI mà không thông báo cho QA. Khi locator hỏng, toàn bộ test suite fail, gây tốn thời gian debug và bảo trì thủ công.

Ví dụ thực tế:

```
# Test viết cho UI phiên bản cũ (v1):
driver.find_element(By.XPATH, '//*[@data-testid="btn-login"]').click()

# Developer nâng cấp lên v2, đổi thành:
# data-testid="login-submit-btn"

# → Selenium ném NoSuchElementException → test FAIL!
```

1.2 Giải Pháp: Self-Healing

Hệ thống Self-Healing hoạt động như một **"lớp đệm thông minh"** giữa test script và UI. Khi locator fail, thay vì dừng test, hệ thống tự động:

- **Phân tích exception** — xác định nguyên nhân fail
- **Quét DOM** — tìm phần tử tương đồng nhất trong trang hiện tại
- **Tính similarity 5 chiều** — attribute, semantic, structure, visual, context
- **Chọn ứng viên tốt nhất** — và tiếp tục chạy test tự động
- **Học và ghi nhớ** — lưu kết quả để dùng lại lần sau

1.3 Kiến Trúc Tổng Quan

Hệ thống gồm 5 module chính phối hợp với nhau:

Module	File	Chức năng
Module 1	Exception_interceptor.py	Bắt và phân loại exception Selenium
Module 2	Snapshot.py	Chụp và lưu fingerprint phần tử HTML
Module 3	Similarity_engine.py	Tính điểm tương đồng 5 chiều
Module 4	Healing_driver.py (CandidateRanker)	Xếp hạng và quyết định chiến lược
Module 5	Healing_driver.py (HealingKnowledgeBase)	Lưu trữ và học từ kết quả healing

```

your_project/
├── backend/
│   ├── core/
│   │   ├── __init__.py
│   │   ├── Snapshot.py           # Module 2: Thu thập fingerprint
│   │   ├── Exception_interceptor.py # Module 1: Xử lý exception
│   │   ├── Similarity_engine.py   # Module 3: Tính similarity
│   │   └── Healing_driver.py      # Module 4+5: Driver chính
│   └── db/
│       └── db_manager.py         # Quản lý database
├── snapshots/                   # Tự động tạo khi chạy
│   └── history/
├── knowledge_base/              # Tự động tạo khi chạy
│   ├── healing_map.json
│   └── healing_log.json
└── tests/
    └── test_login, admin, contact, products, register # Test scripts của hệ thống

```

3. Giải Thích Chi Tiết Từng Module

3.1 Module 1 — Exception Interceptor

File: Exception_interceptor.py

Module này bắt exception từ Selenium và quyết định chiến lược xử lý thay vì crash ngay lập tức. Đây là lớp phòng thủ đầu tiên của hệ thống.

Phân loại exception và chiến lược xử lý:

Exception Selenium	Nguyên nhân	Chiến lược
NoSuchElementException	Locator không tìm thấy → UI đã đổi	TRIGGER_HEALING → kích hoạt full pipeline
StaleElementReferenceException	DOM re-render (React/Vue)	RETRY_FIND → thử lại 3 lần, delay 0.5s
TimeoutException	Trang load chậm	WAIT_AND_RETRY → chờ thêm 10 giây
ElementNotInteractableException	Element bị ẩn/ngoài viewport	SCROLL_INTO_VIEW → JS scrollToView
ElementClickInterceptedException	Overlay/modal che mắt	JS_CLICK → arguments[0].click()

Cách ExceptionInterceptor hoạt động:

```
# Bên trong SelfHealingDriver.find_element():
try:
    element = self._drv.find_element(by, value) # Thử bình thường
except Exception as raw_exc:
    # Module 1 phân tích exception
    result = self._interceptor.intercept(raw_exc, step_name)
    # result.strategy cho biết cần làm gì:
    # - RETRY_FIND → gọi recovery.retry_find(by, value)
    # - WAIT_AND_RETRY → gọi recovery.wait_and_find(by, value)
    # - TRIGGER_HEALING → result.should_heal = True → vào Module 2-5
```

3.2 Module 2 — Snapshot (Thu Thập Fingerprint)

File: Snapshot.py

Module này chụp lại toàn bộ đặc điểm của một phần tử HTML tại thời điểm test pass, lưu thành file JSON để dùng làm cơ sở so sánh khi healing.

ElementSnapshot — Dataclass lưu trữ 5 nhóm dữ liệu:

Nhóm	Các field	Phương pháp thu thập
Attribute	tag, id, classes, name, input_type, placeholder, aria_label, data_testid	Selenium element.get_attribute()
Semantic	text, role, href, value	Selenium element.text, get_attribute()
Structure	xpath_abs, parent_tag, parent_id, depth, sibling_index, siblings_count	JavaScript Executor (hàm đệ quy)

Visual	bounding_x/y/w/h, is_visible, computed_font_size, computed_color	Selenium location/size + JS getComputedStyle()
Context	page_url, page_title, form_context, nearby_label	Selenium driver.current_url + JS DOM traverse

Quy trình thu thập chi tiết của capture_snapshot():

1. Bước 1 — Selenium API đọc attribute:

```
tag      = element.tag_name
el_id    = element.get_attribute('id')
classes  = element.get_attribute('class').split() # → ['btn', 'btn-primary']
data_testid= element.get_attribute('data-testid')
text     = element.text.strip()
```

2. Bước 2 — JavaScript Executor cho Structure (Selenium không có API tương đương):

```
# XPath tuyệt đối — hàm đệ quy leo ngược từ element lên gốc DOM
xpath_abs = driver.execute_script("""
function getXPath(el) {
  if (el.id) return '/' + el.tagName + '[' + '@id=' + el.id + '']';
  if (el === document.body) return '/html/body';
  var pos = 1, siblings = el.parentNode.childNodes;
  for (var i = 0; i < siblings.length; i++) {
    var s = siblings[i];
    if (s === el) return getXPath(el.parentNode) + '/'
      + el.tagName.toLowerCase() + '[' + pos + ']';
    if (s.nodeType === 1 && s.tagName === el.tagName) pos++;
  }
}
return getXPath(arguments[0]);
""", element)

# Depth — đếm số cấp cha leo lên đến <html>
depth = driver.execute_script("""
var d = 0, el = arguments[0];
while (el.parentNode) { d++; el = el.parentNode; }
return d;
""", element)

# Sibling index — vị trí trong danh sách con của cha
sibling_data = driver.execute_script("""
var children = Array.from(arguments[0].parentElement.children);
return { index: children.indexOf(arguments[0]), count: children.length };
""", element)
```

3. Bước 3 — JavaScript cho Visual và Context:

```
# CSS thực tế (computed style) — giá trị browser tính sau khi áp dụng toàn bộ CSS
font_size = driver.execute_script(
  'return window.getComputedStyle(arguments[0]).fontSize;', element)

# Label gần nhất — tìm 2 cách: label[for='id'] hoặc leo ngược DOM
nearby_label = driver.execute_script("""
var el = arguments[0];
if (el.id) {
```

```

    var lbl = document.querySelector('label[for="' + el.id + '"');
    if (lbl) return lbl.textContent.trim();
  }
  var p = el.parentElement;
  while (p) {
    if (p.tagName === 'LABEL') return p.textContent.trim();
    p = p.parentElement;
    if (p && p.tagName === 'FORM') break;
  }
  return null;
", element)

```

4. Bước 4 — Đóng gói và lưu qua SnapshotStore:

```

# Tên file = step_name + ui_version + hash 8 ký tự (unique)
# Ví dụ: email_field_v1_a3f2c1d4.json
store.save(snapshot)

# Tạo 2 file:
# 1. snapshots/email_field_v1_a3f2c1d4.json ← file chính (ghi đè nếu cùng hash)
# 2. snapshots/history/email_field_v1_20260429_135519.json ← audit trail

```

3.3 Module 3 — Similarity Engine (Tính Điểm Tương Đồng)

File: Similarity_engine.py

Module này là trái tim của hệ thống healing. Khi locator fail, module này quét toàn bộ DOM, tính điểm tương đồng giữa snapshot cũ và từng phần tử hiện tại theo 5 chiều.

Luồng hoạt động của find_candidates():

Bước 1 — Lọc ứng viên theo tag:

Chỉ lấy các phần tử cùng tag với original (ví dụ: chỉ lấy <input> nếu original là <input>). Với <button>, quét thêm input[type=submit|button]. Giảm từ hàng trăm xuống còn vài chục phần tử.

Bước 2 — Loại bỏ element ẩn:

```

if not elem.is_displayed(): continue # Bỏ qua hidden elements
if elem.tag_name in ['body', 'html', 'script', 'style']: continue

```

Bước 3 — Tính similarity 5 chiều cho từng ứng viên:

Chi tiết tính điểm từng chiều:

CHIỀU 1 — ATTRIBUTE SCORE

So sánh các thuộc tính HTML của 2 phần tử. Tag khác nhau → nhân hệ số 0.5 (phạt nặng).

```

id      × 0.29 — dùng _str_sim() → SequenceMatcher ratio
classes × 0.22 — dùng Jaccard: |giao| / |hợp|
name    × 0.15 — _str_sim()
input_type × 0.10 — _str_sim()
placeholder × 0.09 — _str_sim()
aria_label × 0.08 — _str_sim()
data_testid × 0.07 — _str_sim()

```

CHIỀU 2 — SEMANTIC SCORE

So sánh ý nghĩa / nội dung của phần tử. Với button, chiều này quan trọng nhất.

text $\times 0.38$ — text hiển thị (Đăng nhập, Submit...)
role $\times 0.22$ — ARIA role (button, textbox...)
href $\times 0.22$ — URL (chỉ dùng cho <a>)
nearby_label $\times 0.13$ — label <label> gần nhất
value $\times 0.05$ — giá trị input hiện tại

CHIỀU 3 — STRUCTURE SCORE

So sánh vị trí trong cây DOM.

xpath_abs $\times 0.26$ — XPath đầy đủ
parent_tag $\times 0.22$ — tag cha (div, form...); khác $\rightarrow 0.2$ (phạt nhẹ)
parent_id $\times 0.18$ — id cha
parent_classes $\times 0.14$ — class cha (Jaccard)
depth $\times 0.08$ — độ sâu DOM (lệch 1 bậc $\rightarrow -0.25$)
sibling_index $\times 0.07$ — vị trí trong cha
siblings_count $\times 0.05$ — tổng số anh em

CHIỀU 4 — VISUAL SCORE

So sánh kích thước và vị trí trên màn hình.

Size similarity (weight 0.6):

w_sim = $\max(0, 1 - |w1 - w2| / \max(w1, 1))$ — width
h_sim = $\max(0, 1 - |h1 - h2| / \max(h1, 1))$ — height

Position similarity (weight 0.4):

x_sim = $\max(0, 1 - |x1 - x2| / \max(x1, 100))$
y_sim = $\max(0, 1 - |y1 - y2| / \max(y1, 100))$

CHIỀU 5 — CONTEXT SCORE

So sánh ngữ cảnh trang và form chứa phần tử.

form_context $\times 0.70$ — id/class của <form> cha gần nhất
page_url $\times 0.30$ — URL trang (chỉ so sánh path, bỏ query string)

5. Bước 4 — Tính tổng điểm có trọng số và lọc:

```
total = (attribute_score * weights['attribute'] +  
        semantic_score * weights['semantic'] +  
        structure_score * weights['structure'] +
```

```

visual_score * weights['visual'] +
context_score * weights['context'])

# Chỉ giữ ứng viên có score >= 0.40 (threshold lọc sơ bộ)
if similarity.total_score >= 0.40:
    candidates.append(Candidate(element, cand_snap, similarity))

# Sắp xếp giảm dần — ứng viên tốt nhất ở đầu danh sách
candidates.sort(key=lambda c: c.score, reverse=True)

```

Bộ trọng số:

(model): 'attribute': 0.19, 'semantic': 0.19, 'structure': 0.07, 'visual': 0.54, 'context': 0.01
(saw/ahp)'attribute': 0.28, 'semantic': 0.23, 'structure': 0.2, 'visual': 0.17, 'context': 0.12

3.4 Module 4 — Candidate Ranker (Xếp Hạng và Quyết Định)

Class: CandidateRanker trong Healing_driver.py

Module này nhận danh sách ứng viên từ Module 3 và quyết định hành động tiếp theo dựa trên ngưỡng tin cậy.

Hệ thống ngưỡng quyết định:

Ngưỡng	Giá trị	Hành động	Ý nghĩa
THRESHOLD_AUTO	≥ 0.75	AUTO_FIX	Tự động thay locator, test tiếp tục chạy
THRESHOLD_SUGGEST	0.50 – 0.75	SUGGEST	Tìm được ứng viên nhưng không chắc, chỉ gợi ý
FAILED	< 0.50	FAILED	Không tìm được ứng viên tin cậy, raise exception

Chọn locator bền vững nhất (get_best_locator):

Sau khi chọn được ứng viên, hệ thống ưu tiên locator bền vững nhất theo thứ tự:

Ưu tiên	Locator	Lý do
1 (cao nhất)	data-testid	Attribute dành riêng cho testing, ít bị đổi nhất
2	aria-label	Attribute accessibility, tương đối ổn định
3	name	Attribute form, thường ổn định
4	placeholder	Thay đổi ít khi refactor
5	id	Thường ổn nhưng đôi khi bị đổi
6 (thấp nhất)	XPath theo text	Chỉ dùng cho button/link khi không có gì khác

3.5 Module 5 — Knowledge Base (Lưu Trữ và Học)

Class: HealingKnowledgeBase trong Healing_driver.py

Module này ghi nhớ kết quả healing để tái sử dụng, tránh phải tính toán lại từ đầu mỗi lần.

Cấu trúc file lưu trữ:

```
knowledge_base/  
├── healing_map.json  ← tra cứu nhanh: locator cũ → locator mới  
└── healing_log.json  ← lịch sử toàn bộ healing events
```

healing_map.json — cấu trúc key-value:

```
{  
  "email_field:v2::css selector::[data-testid=\"login-email\"]": {  
    "step_name": "email_field",  
    "ui_version": "v2",  
    "old_locator_type": "css selector",  
    "old_locator_value": "[data-testid=\"login-email\"]",  
    "new_locator_type": "xpath",  
    "new_locator_value": "///input[@data-testid='email-field']",  
    "confidence": 0.9487,  
    "similarity_detail": {  
      "attribute_score": 0.8036,  
      "semantic_score": 1.0,  
      "structure_score": 0.8,  
      "visual_score": 1.0,  
      "context_score": 1.0,  
      "total_score": 0.9487,  
      "weights_used": {  
        "attribute": 0.19,  
        "semantic": 0.19,  
        "structure": 0.07,  
        "visual": 0.54,  
        "context": 0.01  
      },  
    },  
    "attribute_detail": {  
      "tag": 1.0,  
      "id": 0.455,  
      "classes": 1.0,  
      "name": 1.0,  
      "input_type": 1.0,  
      "placeholder": 1.0,  
      "aria_label": 1.0,  
      "data_testid": 0.455  
    },  
    "semantic_detail": {  
      "text": 1.0,  
      "role": 1.0,  
      "href": 1.0,  
      "nearby_label": 1.0,  
      "value": 1.0  
    },  
    "structure_detail": {  
      "xpath": 0.769,  
      "parent_tag": 1.0,  
      "parent_id": 1.0,  
    },  
  },  
}
```

```

    "parent_classes": 0.0,
    "depth": 1.0,
    "sibling_index": 1.0,
    "siblings_count": 1.0
  },
  "dynamic_flags": {
    "id": false,
    "data_testid": false,
    "classes": false
  }
},
"learned_at": "2026-05-06T13:25:58.923489",
"times_used": 32
}}

```

Luồng tra cứu cache:

```

# Trước khi chạy full healing pipeline, kiểm tra cache trước
cached = self._kb.lookup(step_name, str(by), value, ui_version)
if cached:
    new_by, new_val = cached
    element = self._drv.find_element(new_by, new_val) # Dùng locator đã học
    self._kb.increment_usage(step_name, str(by), value, ui_version)
    return element # Không cần chạy ML, nhanh hơn nhiều

# Nếu không có trong cache → chạy full healing pipeline

```

4. Luồng Chạy Toàn Bộ Hệ Thống

Để chạy hệ thống:

B1: Vào terminal PowerShell chạy lệnh ``cd backend`` (để vào thư mục backend) sau đó chạy lệnh ``node server.js`` (để kết nối với database) .

B2: Mở 1 terminal PowerShell mới chạy lệnh ``cd myapp`` (để vào thư mục chứa các file JSX để chạy web) .

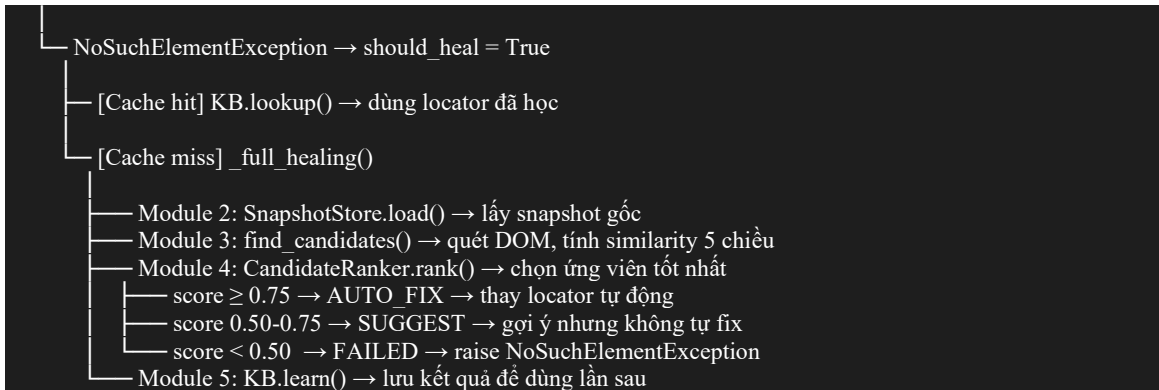
B3: Mở 1 terminal bash(MSYS2) chạy lệnh ``pytest tests/*file muốn test ( test_login, test_admin, test_contact, test_products, test_register*)`` để kích hoạt hệ thống self healing. (Nếu không có cần cài đặt)

4.1 Sơ Đồ Luồng find_element()

```

driver.find_element(by, value, step_name, ui_version)
├── [Thành công]
│   ├── capture_snapshot() → lưu fingerprint
│   └── trả về element
├── [Thất bại] → ExceptionInterceptor.intercept(exception)
│   ├── StaleElementReferenceException
│   │   └── RecoveryActions.retry find() × 3 lần, delay 0.5s
│   ├── TimeoutException
│   │   └── RecoveryActions.wait_and_find() → WebDriverWait 10s
│   ├── ElementNotInteractableException
│   │   └── [xử lý bên ngoài] scroll_into_view()
│   └── ElementClickInterceptedException
│       └── [xử lý bên ngoài] js_click()

```



4.2 Vòng Đòi Dữ Liệu Của Hệ Thống

LẦN ĐẦU CHẠY (UI v1):

test pass → capture_snapshot() → lưu snapshots/email_field_v1_xxxx.json

UI NÂNG CẤP LÊN v2 (locator thay đổi):

test fail → ExceptionInterceptor: NoSuchElementException

→ KB cache miss

→ load snapshot v1 từ file

→ find_candidates(): quét 15 input trên trang → tính similarity

→ CandidateRanker: best score = 0.959 → AUTO_FIX

→ KB.learn(): lưu 'email-field' → 'email-input-v2' vào healing_map.json

→ test tiếp tục chạy

LẦN CHẠY SAU (cùng v2):

test fail (cùng locator cũ)

→ KB.lookup(): cache hit!

→ dùng ngay locator đã học, không cần chạy heal lại

→ test tiếp tục (nhanh hơn ~10x)

5. Module Máy Học — Tối Ưu Trọng Số

5.1 Mục Đích

File: train_model.ipynb

Mặc định, hệ thống dùng trọng số cứng cho 5 chiều similarity. Sau khi tích lũy đủ dữ liệu healing, có thể chạy notebook này để học trọng số tối ưu từ dữ liệu thực tế — giúp hệ thống ngày càng chính xác hơn.

5.2 Cách Chạy

- ✓ Đảm bảo có file candidate_scores.csv (tự động sinh trong quá trình chạy test).
- ✓ Mở file train_model.ipynb bằng Jupyter Notebook hoặc VS Code.
- ✓ Chạy từng cell theo thứ tự từ đầu đến cuối.
- ✓ Kết quả: bộ trọng số tối ưu được in ra ở cell cuối.
- ✓ Cập nhật trọng số vào DEFAULT_WEIGHTS trong Similarity_engine.py.

5.3 Kết Quả Thực Nghiệm

Trọng số được học từ 407 mẫu dữ liệu thực tế (64 healing events train, 16 test):

Feature	Trọng số học được	Trọng số mặc định	Nhận xét
visual_score	53.9%	17%	Cao hơn kỳ vọng — phần tử giữ vị trí trực quan
attr_score	19.1%	28%	Thấp hơn do attribute thay đổi khi refactor
sem_score	18.9%	23%	Semantic ổn định theo thời gian
struct_score	7.5%	20%	Cấu trúc DOM thay đổi khi redesign
ctx_score	0.6%	12%	Context ít phân biệt trong dataset nhỏ

Nhận xét:

Mặc dù các nghiên cứu trước thường đánh giá attribute, semantic và structure là các feature quan trọng nhất, kết quả Logistic Regression trên dữ liệu thực nghiệm cho thấy visual similarity có trọng số cao nhất. Nguyên nhân chủ yếu đến từ đặc điểm mutation trong dataset hiện tại, nơi nhiều phần tử thay đổi thuộc tính định danh nhưng vẫn giữ vị trí trực quan tương đối ổn định. Ngoài ra, hiện tượng multicollinearity giữa attribute, semantic và structure khiến mô hình có xu hướng dồn trọng số vào một feature đại diện mạnh hơn. Do đó kết quả này phản ánh hành vi thực tế của dữ liệu.

5.4 Hiệu Quả Mô Hình

Chỉ số	Giá trị	Nhận xét
Accuracy	96.23%	Phân loại đúng 96% mẫu
Recall (positive)	100%	Không bỏ sót ứng viên đúng nào
F1-Score	88.89%	Cân bằng tốt giữa precision và recall
ROC-AUC	99.03%	Khả năng phân biệt winner/loser tốt
Top-1 Success Rate	100%	Ứng viên đúng luôn được xếp hạng cao nhất

6. Ưu/nhược điểm của hệ thống

6.1 Khi Nào Hệ Thống Hoạt Động Tốt

- **Thay đổi locator/attribute** — đổi data-testid, id, class → heal được
- **Đổi toàn bộ kiểu locator** — từ data-testid sang aria-label → heal được (semantic score vẫn cao)
- **Thay đổi layout nhỏ** — button dịch sang cột khác → heal được (visual chỉ weight 10%)
- **DOM re-render** — React/Vue re-render → xử lý bằng StaleElement retry
- **Trang load chậm** — xử lý bằng WebDriverWait 10s

6.2 Giới Hạn Của Hệ Thống

- **Redesign hoàn toàn** — đổi toàn bộ tag, text, vị trí, context → FAILED (đúng behavior)
- **Xóa tính năng** — phần tử không còn tồn tại → FAILED (đây là bug thật, không nên heal)
- **Dataset nhỏ** — trọng số ML học từ 80 events, cần thêm dữ liệu để cải thiện

7. Tổng Kết

Hệ thống Self-Healing Test cung cấp một giải pháp toàn diện để giải quyết vấn đề locator bị hỏng trong Selenium automation:

Tính năng	Mô tả
Tự động phục hồi	Xử lý 5 loại exception Selenium phổ biến nhất
Similarity 5 chiều	So sánh attribute, semantic, structure, visual, context
Cache thông minh	Lưu kết quả healing, dùng lại không cần tính lại
Học từ dữ liệu	Logistic Regression tối ưu trọng số từ lịch sử healing
Đễ tích hợp	Chỉ thay 2 dòng code, không cần thay đổi test logic